In [5]:
```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC

from xgboost import XGBClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)

import warnings
warnings.filterwarnings("ignore")
```

In [6]:
```
df = pd.read_excel("Smart_Shoe_Gait_Dataset.xlsx")

print("Dataset loaded successfully!")
```

```
Dataset loaded successfully!
```

In [7]:
```
df.head()
```

Out[7]:

	Patient_ID	Age	Gender	Height_cm	Weight_kg	BMI	Heel_Pressure_kPa	Midfoot_Pressure_kPa	Toe_Pressure_
0	P0001	8	Female	123	31	20.5	114	134	
1	P0002	3	Male	98	13	13.5	213	129	
2	P0003	4	Female	96	17	18.4	143	212	
3	P0004	5	Female	112	18	14.3	192	214	
4	P0005	8	Male	132	31	17.8	112	145	

In [8]:
```
print("Missing Values:")
print(df.isnull().sum())
```

```
Missing Values:
Patient_ID               0
Age                      0
Gender                   0
Height_cm                0
Weight_kg                0
BMI                      0
Heel_Pressure_kPa        0
Midfoot_Pressure_kPa     0
Toe_Pressure_kPa         0
Total_Pressure_kPa       0
Dominant_Zone            0
Walking_Speed_mps        0
Gait_Type                0
Disease                408
Risk_Level               0
Recommendation           0
dtype: int64
```

In [9]:
```
df[df["Disease"].isnull()]["Gait_Type"].value_counts()
```

Out[9]:
```
Gait_Type
Normal    408
Name: count, dtype: int64
```

In [10]:
```
pd.crosstab(
    df["Gait_Type"],
    df["Disease"].isnull()
)
```

Out[10]:

Disease	False	True
Gait_Type		
Abnormal	241	0
Flat Foot	93	0
Heel Dominant	141	0
Normal	0	408
Toe Dominant	117	0

In [11]:
```
df.describe()
```

Out[11]:

	Age	Height_cm	Weight_kg	BMI	Heel_Pressure_kPa	Midfoot_Pressure_kPa	Toe_Pressur
count	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.0
mean	5.522000	110.645000	21.136000	17.068500	147.034000	150.319000	26.6
std	1.676403	10.843272	5.291834	2.297328	35.010721	28.529549	24.2
min	3.000000	90.000000	13.000000	12.600000	100.000000	120.000000	0.0
25%	4.000000	102.000000	17.000000	15.300000	118.000000	130.000000	12.0
50%	6.000000	110.000000	20.000000	17.000000	138.000000	139.000000	19.0
75%	7.000000	120.000000	24.000000	18.500000	177.250000	167.250000	25.0
max	8.000000	132.000000	35.000000	24.300000	220.000000	220.000000	100.0

In [12]:
```
gait_counts = df["Gait_Type"].value_counts()

print(gait_counts)
```

```
Gait_Type
Normal           408
Abnormal         241
Heel Dominant    141
Toe Dominant     117
Flat Foot         93
Name: count, dtype: int64
```

In [13]:
```
gait_percentage = (
    df["Gait_Type"]
    .value_counts(normalize=True)
    .mul(100)
    .round(2)
)

print(gait_percentage)
```

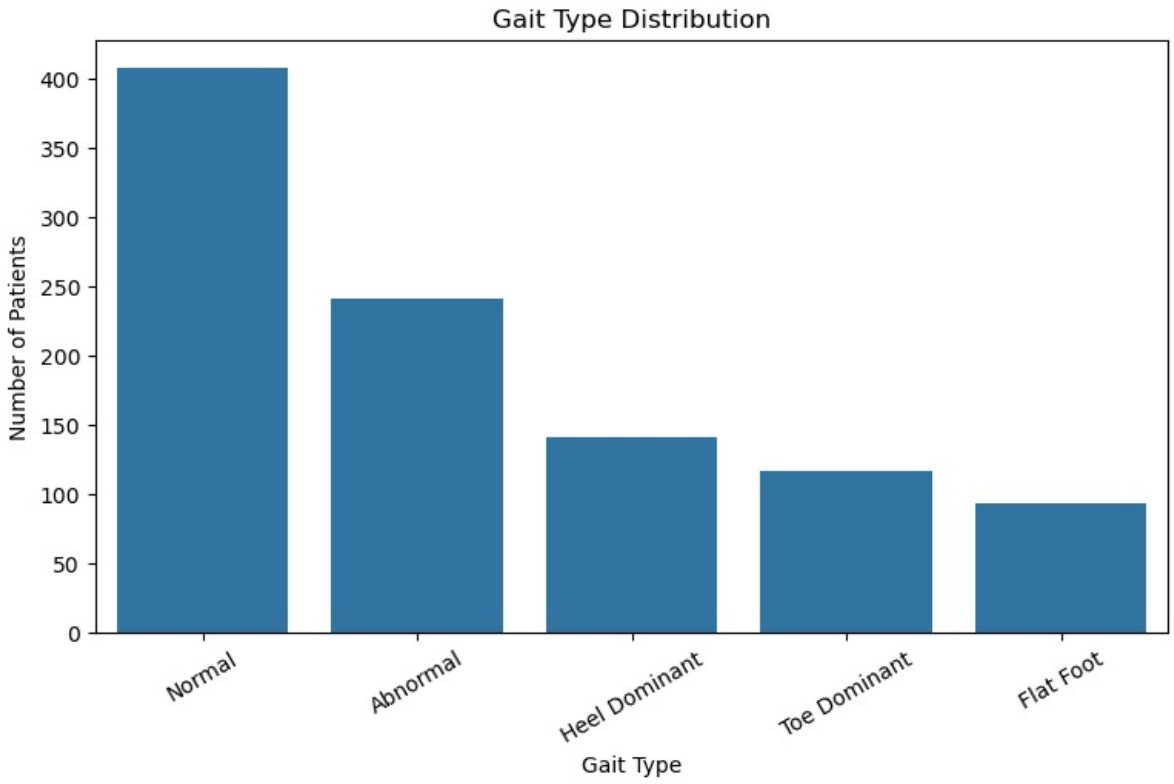
```
Gait_Type
Normal           40.8
Abnormal         24.1
Heel Dominant    14.1
Toe Dominant     11.7
Flat Foot         9.3
Name: proportion, dtype: float64
```

In [14]:
```
plt.figure(figsize=(9, 5))

sns.countplot(
    data=df,
    x="Gait_Type",
    order=df["Gait_Type"].value_counts().index
)

plt.title("Gait Type Distribution")
plt.xlabel("Gait Type")
plt.ylabel("Number of Patients")
plt.xticks(rotation=30)
```

```
plt.show()
```



In [15]:
```
pressure_columns = [
    "Heel_Pressure_kPa",
    "Midfoot_Pressure_kPa",
    "Toe_Pressure_kPa"
]

df[pressure_columns].describe()
```

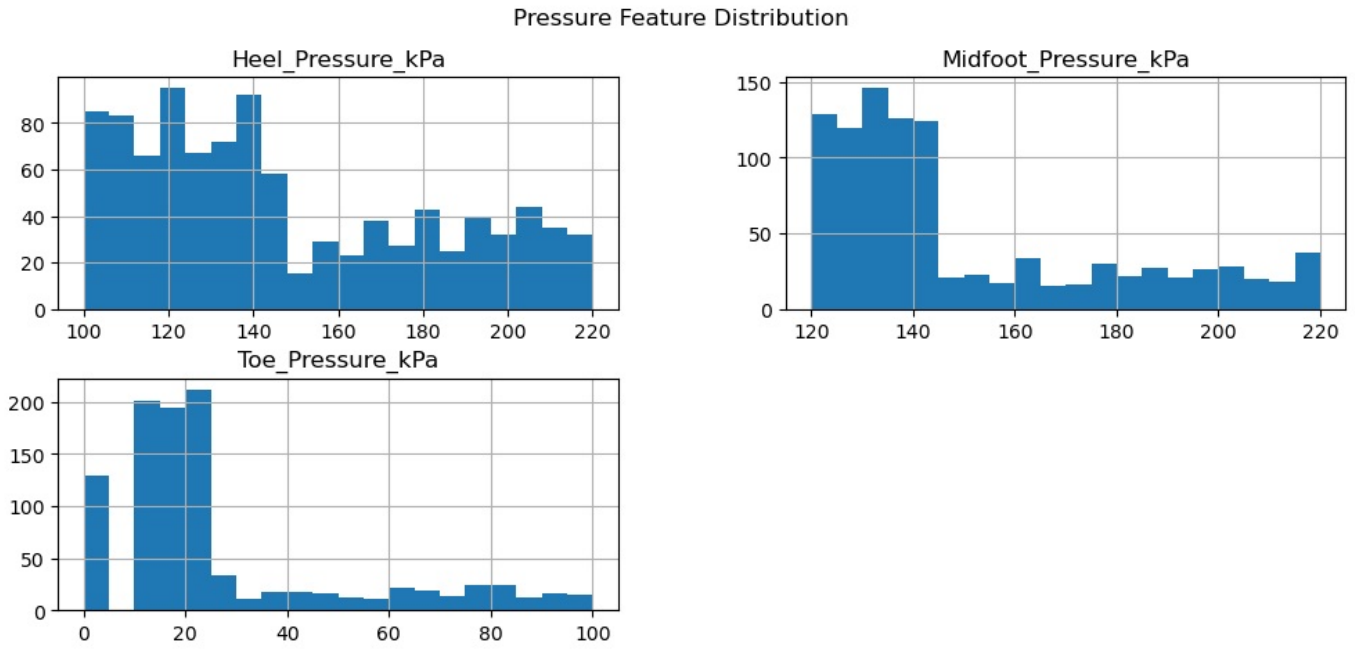
Out[15]:

	Heel_Pressure_kPa	Midfoot_Pressure_kPa	Toe_Pressure_kPa
count	1000.000000	1000.000000	1000.000000
mean	147.034000	150.319000	26.634000
std	35.010721	28.529549	24.201689
min	100.000000	120.000000	0.000000
25%	118.000000	130.000000	12.000000
50%	138.000000	139.000000	19.000000
75%	177.250000	167.250000	25.000000
max	220.000000	220.000000	100.000000

In [16]:
```
df[pressure_columns].hist(
    figsize=(12, 5),
    bins=20
)

plt.suptitle("Pressure Feature Distribution")

plt.show()
```

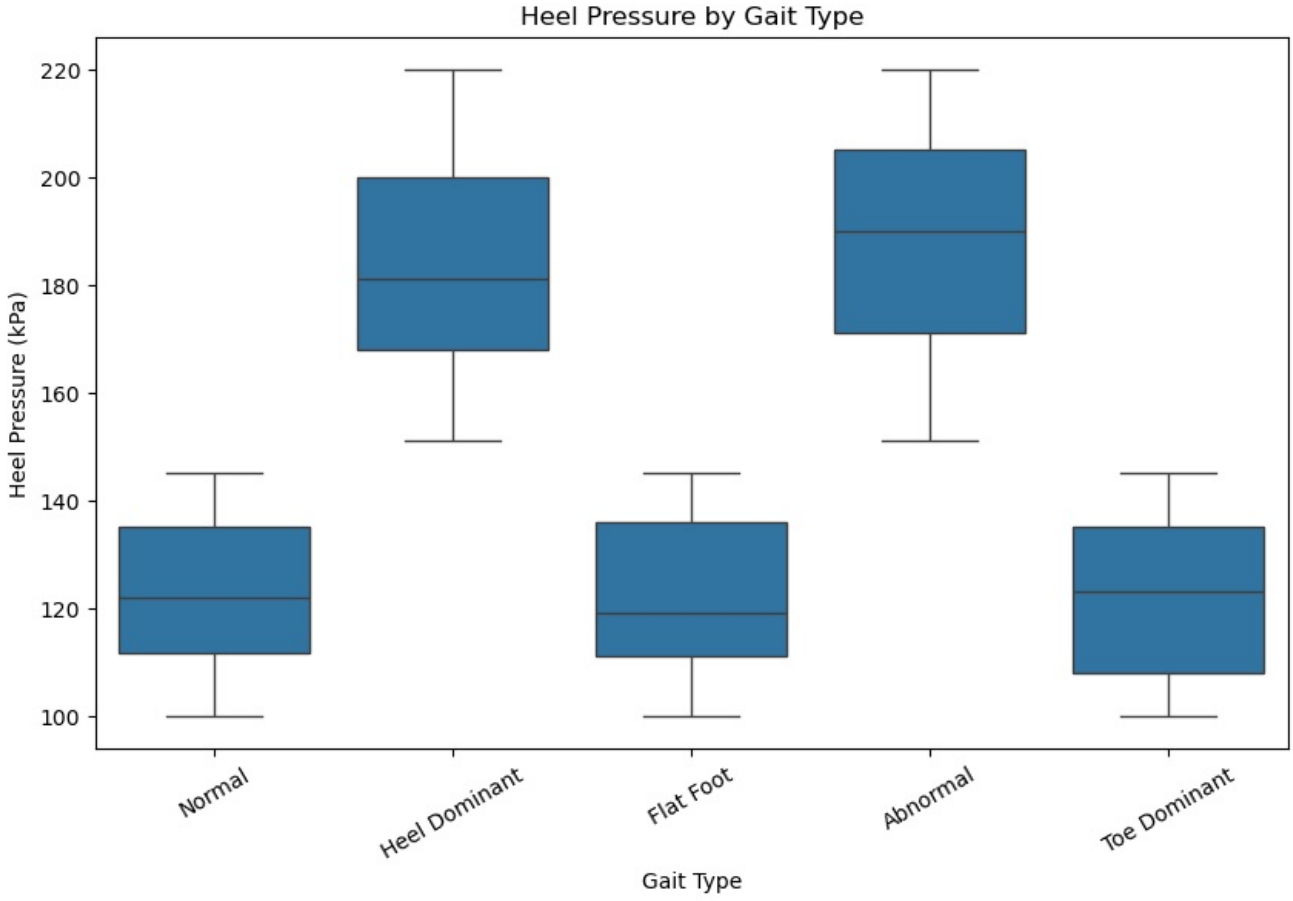


In [17]:
```
plt.figure(figsize=(10, 6))

sns.boxplot(
    data=df,
    x="Gait_Type",
    y="Heel_Pressure_kPa"
)

plt.title("Heel Pressure by Gait Type")
plt.xlabel("Gait Type")
plt.ylabel("Heel Pressure (kPa)")
plt.xticks(rotation=30)

plt.show()
```



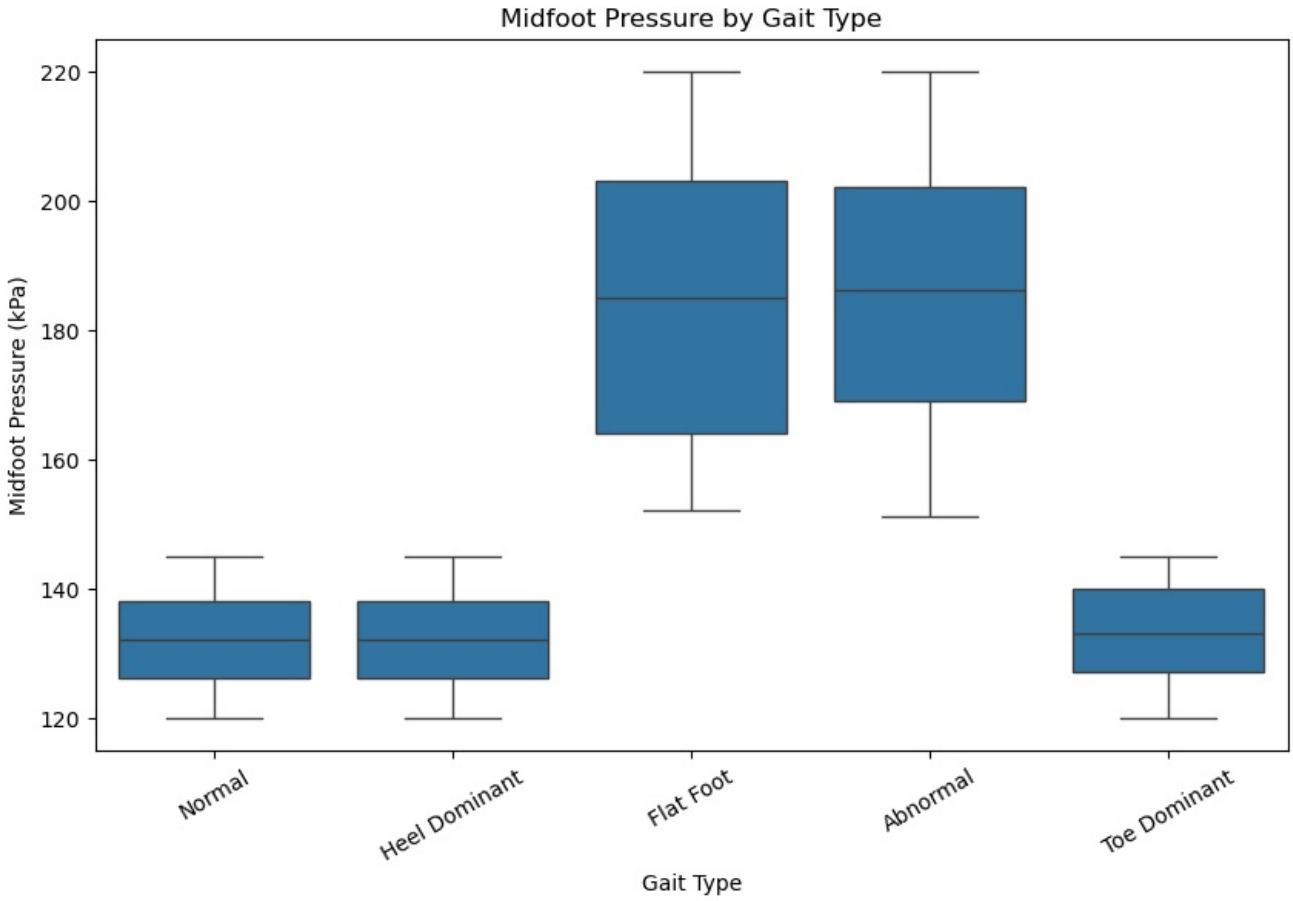
In [18]:
```
plt.figure(figsize=(10, 6))

sns.boxplot(
    data=df,
    x="Gait_Type",
    y="Midfoot_Pressure_kPa"
```

```
)

plt.title("Midfoot Pressure by Gait Type")
plt.xlabel("Gait Type")
plt.ylabel("Midfoot Pressure (kPa)")
plt.xticks(rotation=30)

plt.show()
```



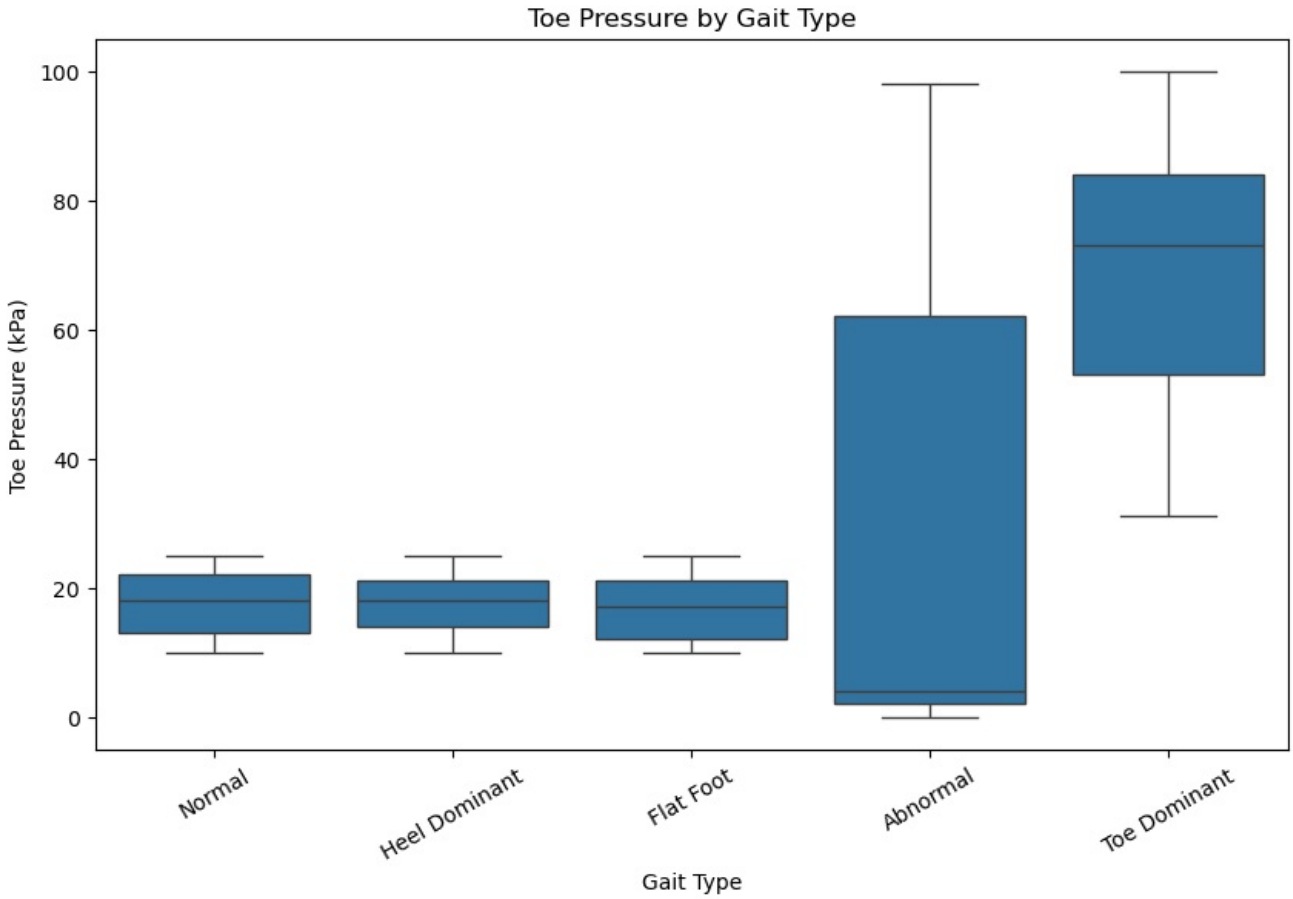
In [19]:

```
plt.figure(figsize=(10, 6))

sns.boxplot(
    data=df,
    x="Gait_Type",
    y="Toe_Pressure_kPa"
)

plt.title("Toe Pressure by Gait Type")
plt.xlabel("Gait Type")
plt.ylabel("Toe Pressure (kPa)")
plt.xticks(rotation=30)

plt.show()
```

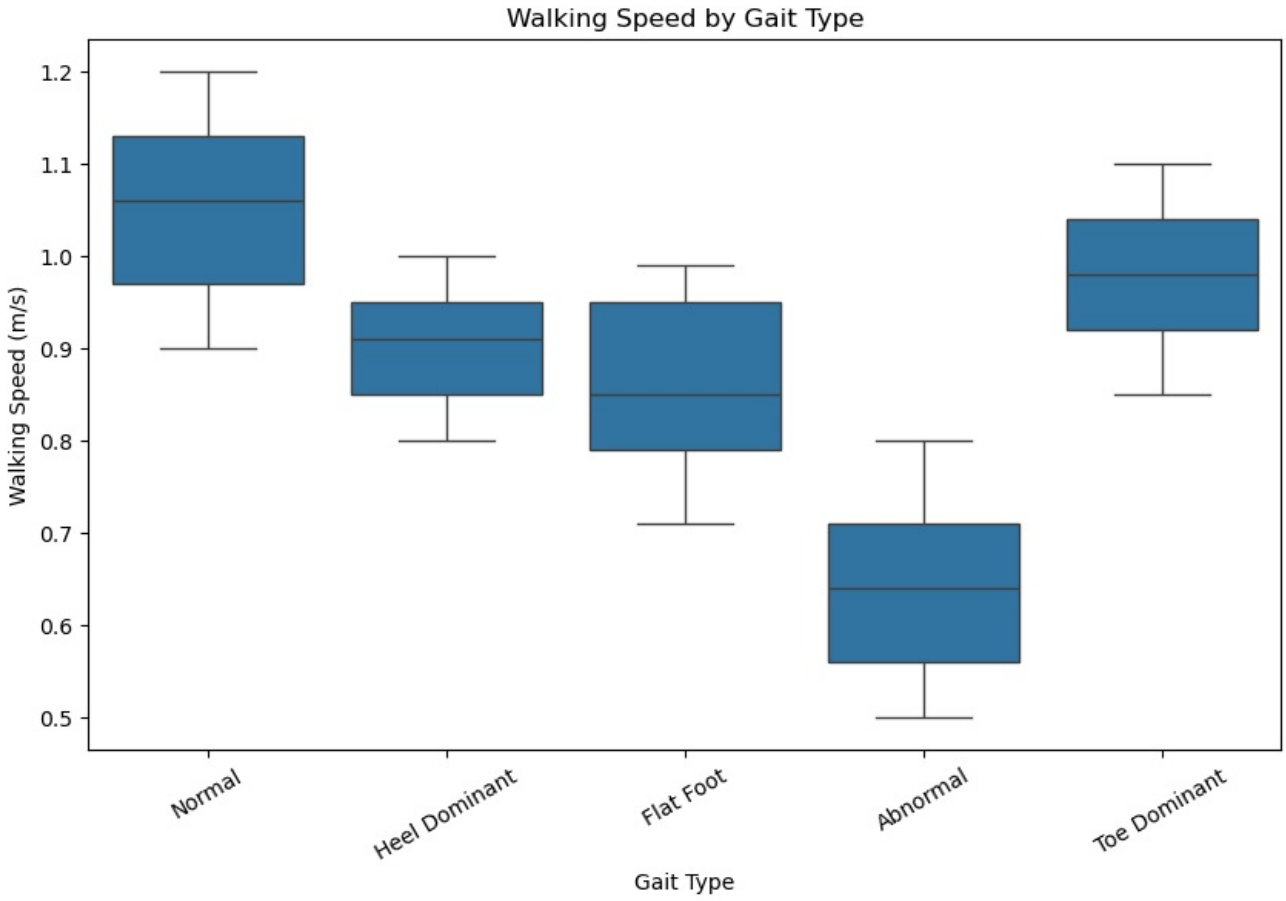


In [20]:
```
plt.figure(figsize=(10, 6))

sns.boxplot(
    data=df,
    x="Gait_Type",
    y="Walking_Speed_mps"
)

plt.title("Walking Speed by Gait Type")
plt.xlabel("Gait Type")
plt.ylabel("Walking Speed (m/s)")
plt.xticks(rotation=30)

plt.show()
```



In [21]:

```
numeric_columns = [
    "Age",
    "Height_cm",
    "Weight_kg",
    "BMI",
    "Heel_Pressure_kPa",
    "Midfoot_Pressure_kPa",
    "Toe_Pressure_kPa",
    "Total_Pressure_kPa",
    "Walking_Speed_mps"
]

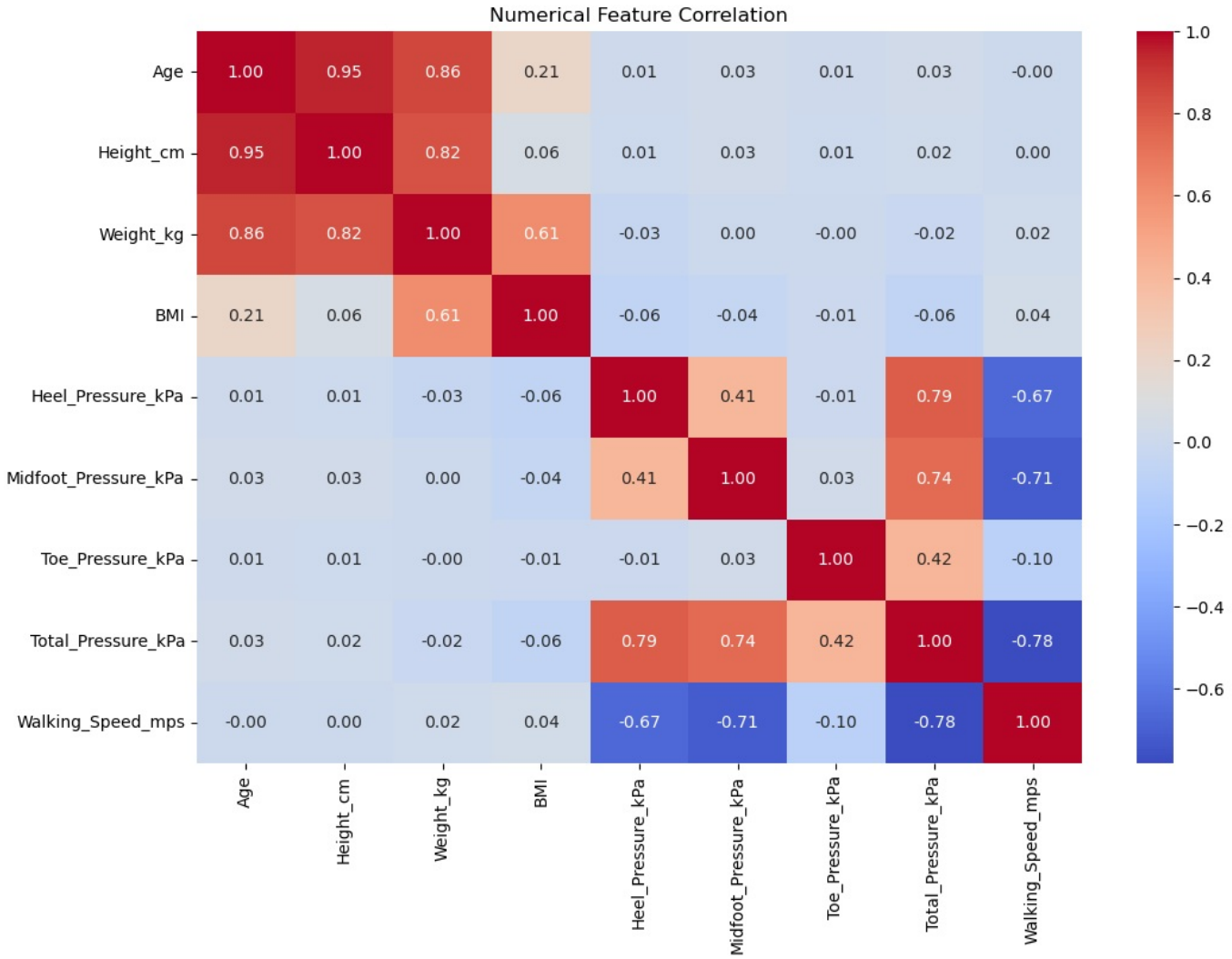
correlation = df[numeric_columns].corr()

plt.figure(figsize=(12, 8))

sns.heatmap(
    correlation,
    annot=True,
    fmt=".2f",
    cmap="coolwarm"
)

plt.title("Numerical Feature Correlation")

plt.show()
```



In [22]:
```
df["Heel_Ratio"] = (
    df["Heel_Pressure_kPa"] /
    df["Total_Pressure_kPa"]
)

df["Midfoot_Ratio"] = (
    df["Midfoot_Pressure_kPa"] /
    df["Total_Pressure_kPa"]
)

df["Toe_Ratio"] = (
    df["Toe_Pressure_kPa"] /
    df["Total_Pressure_kPa"]
)
```

In [23]:
```
df[
    [
        "Heel_Pressure_kPa",
        "Midfoot_Pressure_kPa",
        "Toe_Pressure_kPa",
        "Total_Pressure_kPa",
        "Heel_Ratio",
        "Midfoot_Ratio",
        "Toe_Ratio"
    ]
].head()
```

Out[23]:

	Heel_Pressure_kPa	Midfoot_Pressure_kPa	Toe_Pressure_kPa	Total_Pressure_kPa	Heel_Ratio	Midfoot_Ratio	To
0	114	134	17	265	0.430189	0.505660	0
1	213	129	22	364	0.585165	0.354396	0
2	143	212	20	375	0.381333	0.565333	0
3	192	214	1	407	0.471744	0.525799	0
4	112	145	15	272	0.411765	0.533088	0

In [24]:
```
features = [
```

```
    "Age",
    "Gender",
    "Height_cm",
    "Weight_kg",
    "BMI",
    "Heel_Pressure_kPa",
    "Midfoot_Pressure_kPa",
    "Toe_Pressure_kPa",
    "Total_Pressure_kPa",
    "Dominant_Zone",
    "Walking_Speed_mps",
    "Heel_Ratio",
    "Midfoot_Ratio",
    "Toe_Ratio"
]

X = df[features].copy()

y = df["Gait_Type"].copy()

print("X Shape:", X.shape)
print("y Shape:", y.shape)
```

```
X Shape: (1000, 14)
y Shape: (1000,)
```

In [25]:
```
X = pd.get_dummies(
    X,
    columns=[
        "Gender",
        "Dominant_Zone"
    ],
    drop_first=False
)

print("Encoded Feature Shape:", X.shape)
```

```
Encoded Feature Shape: (1000, 16)
```

In [26]:
```
label_encoder = LabelEncoder()

y_encoded = label_encoder.fit_transform(y)

print("Gait Classes:")
print(label_encoder.classes_)
```

```
Gait Classes:
['Abnormal' 'Flat Foot' 'Heel Dominant' 'Normal' 'Toe Dominant']
```

In [27]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y_encoded,
    test_size=0.20,
    random_state=42,
    stratify=y_encoded
)

print("Training Samples:", X_train.shape[0])
print("Testing Samples:", X_test.shape[0])
```

```
Training Samples: 800
Testing Samples: 200
```

In [28]:
```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)

print("Feature scaling completed!")
```

```
Feature scaling completed!
```

In [29]:
```
logistic_model = LogisticRegression(
    max_iter=1000,
    random_state=42
)

logistic_model.fit(
    X_train_scaled,
    y_train
)

y_pred_logistic = logistic_model.predict(
    X_test_scaled
```

```
)

print("Logistic Regression trained successfully!")
```

```
Logistic Regression trained successfully!
```

In [30]:
```
dt_model = DecisionTreeClassifier(
    max_depth=5,
    random_state=42
)

dt_model.fit(
    X_train,
    y_train
)

y_pred_dt = dt_model.predict(X_test)

print("Decision Tree trained successfully!")
```

```
Decision Tree trained successfully!
```

In [31]:
```
rf_model = RandomForestClassifier(
    n_estimators=200,
    random_state=42
)

rf_model.fit(
    X_train,
    y_train
)

y_pred_rf = rf_model.predict(X_test)

print("Random Forest trained successfully!")
```

```
Random Forest trained successfully!
```

In [32]:
```
svm_model = SVC(
    kernel="rbf",
    probability=True,
    random_state=42
)

svm_model.fit(
    X_train_scaled,
    y_train
)

y_pred_svm = svm_model.predict(
    X_test_scaled
)

print("SVM trained successfully!")
```

```
SVM trained successfully!
```

In [33]:
```
xgb_model = XGBClassifier(
    n_estimators=200,
    max_depth=5,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    objective="multi:softmax",
    num_class=len(label_encoder.classes_),
    eval_metric="mlogloss",
    random_state=42
)

xgb_model.fit(
    X_train,
    y_train
)

y_pred_xgb = xgb_model.predict(X_test)

print("XGBoost trained successfully!")
```

```
XGBoost trained successfully!
```

In [34]:
```
def evaluate_model(model_name, y_true, y_pred):

    accuracy = accuracy_score(
        y_true,
        y_pred
```

```
    )

    precision = precision_score(
        y_true,
        y_pred,
        average="weighted",
        zero_division=0
    )

    recall = recall_score(
        y_true,
        y_pred,
        average="weighted",
        zero_division=0
    )

    f1 = f1_score(
        y_true,
        y_pred,
        average="weighted",
        zero_division=0
    )

    return {
        "Model": model_name,
        "Accuracy": accuracy,
        "Precision": precision,
        "Recall": recall,
        "F1-Score": f1
    }
```

In [35]:

```
results = []

results.append(
    evaluate_model(
        "Logistic Regression",
        y_test,
        y_pred_logistic
    )
)

results.append(
    evaluate_model(
        "Decision Tree",
        y_test,
        y_pred_dt
    )
)

results.append(
    evaluate_model(
        "Random Forest",
        y_test,
        y_pred_rf
    )
)

results.append(
    evaluate_model(
        "SVM",
        y_test,
        y_pred_svm
    )
)

results.append(
    evaluate_model(
        "XGBoost",
        y_test,
        y_pred_xgb
    )
)

results_df = pd.DataFrame(results)

results_df
```

Out[35]:

	Model	Accuracy	Precision	Recall	F1-Score
0	Logistic Regression	0.995	0.995060	0.995	0.994960
1	Decision Tree	0.995	0.995208	0.995	0.995038
2	Random Forest	1.000	1.000000	1.000	1.000000
3	SVM	0.990	0.990238	0.990	0.989833
4	XGBoost	1.000	1.000000	1.000	1.000000

In [36]:

```
results_percentage = results_df.copy()

metric_columns = [
    "Accuracy",
    "Precision",
    "Recall",
    "F1-Score"
]

results_percentage[metric_columns] = (
    results_percentage[metric_columns] * 100
).round(2)

results_percentage
```

Out[36]:

	Model	Accuracy	Precision	Recall	F1-Score
0	Logistic Regression	99.5	99.51	99.5	99.50
1	Decision Tree	99.5	99.52	99.5	99.50
2	Random Forest	100.0	100.00	100.0	100.00
3	SVM	99.0	99.02	99.0	98.98
4	XGBoost	100.0	100.00	100.0	100.00

In [37]:

```
best_index = results_df["F1-Score"].idxmax()

best_model_name = results_df.loc[
    best_index,
    "Model"
]

print("Best Model:", best_model_name)

print(
    "Accuracy:",
    round(
        results_df.loc[best_index, "Accuracy"] * 100,
        2
    ),
    "%"
)

print(
    "F1-Score:",
    round(
        results_df.loc[best_index, "F1-Score"] * 100,
        2
    ),
    "%"
)
```

```
Best Model: Random Forest
Accuracy: 100.0 %
F1-Score: 100.0 %
```

In [38]:

```
plt.figure(figsize=(10, 6))

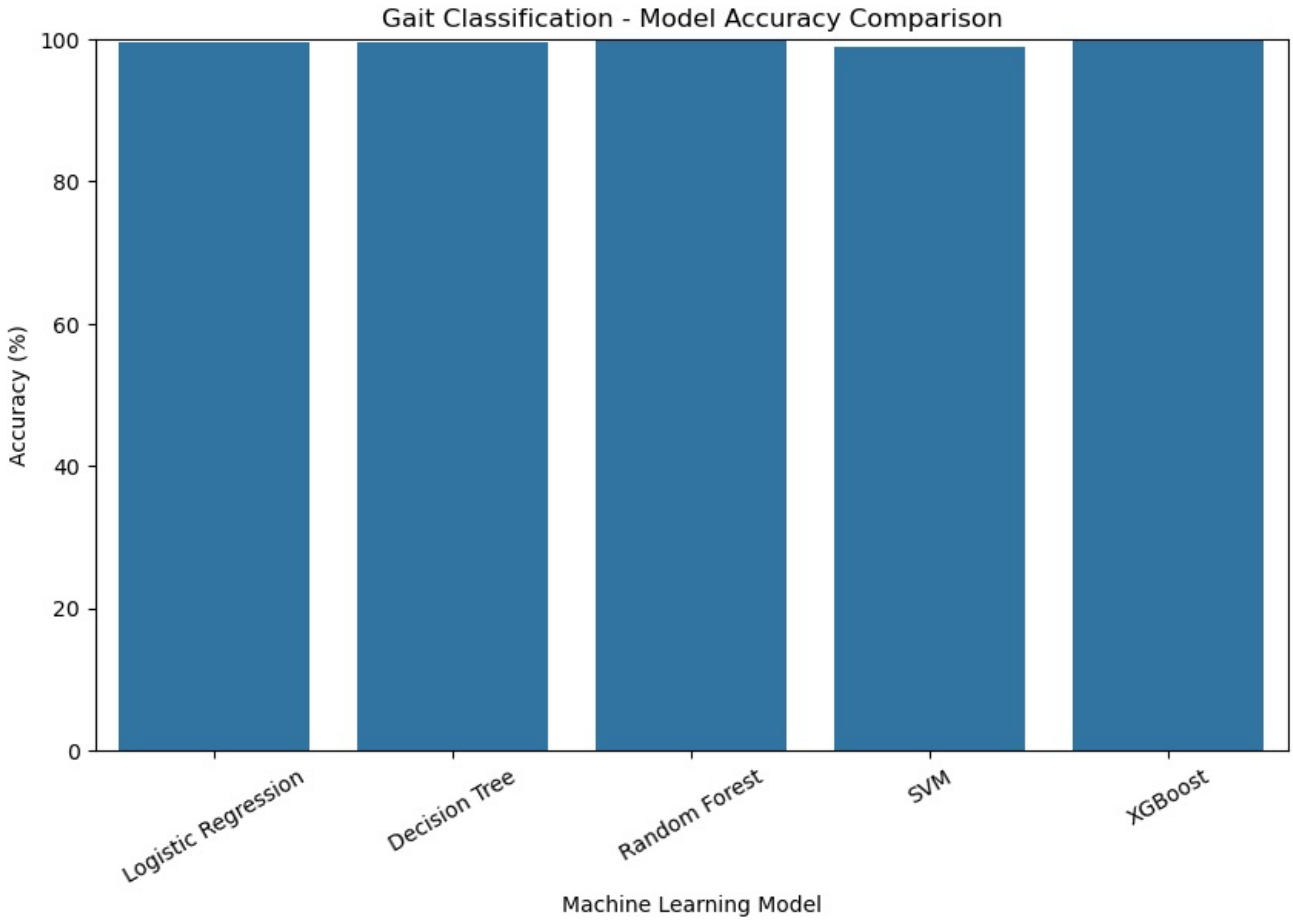
sns.barplot(
    data=results_percentage,
    x="Model",
    y="Accuracy"
)

plt.title("Gait Classification - Model Accuracy Comparison")
plt.xlabel("Machine Learning Model")
plt.ylabel("Accuracy (%)")

plt.ylim(0, 100)
```

```
plt.xticks(rotation=30)

plt.show()
```



In [39]:
```
metrics_long = results_percentage.melt(
    id_vars="Model",
    value_vars=[
        "Accuracy",
        "Precision",
        "Recall",
        "F1-Score"
    ],
    var_name="Metric",
    value_name="Score"
)

plt.figure(figsize=(12, 6))

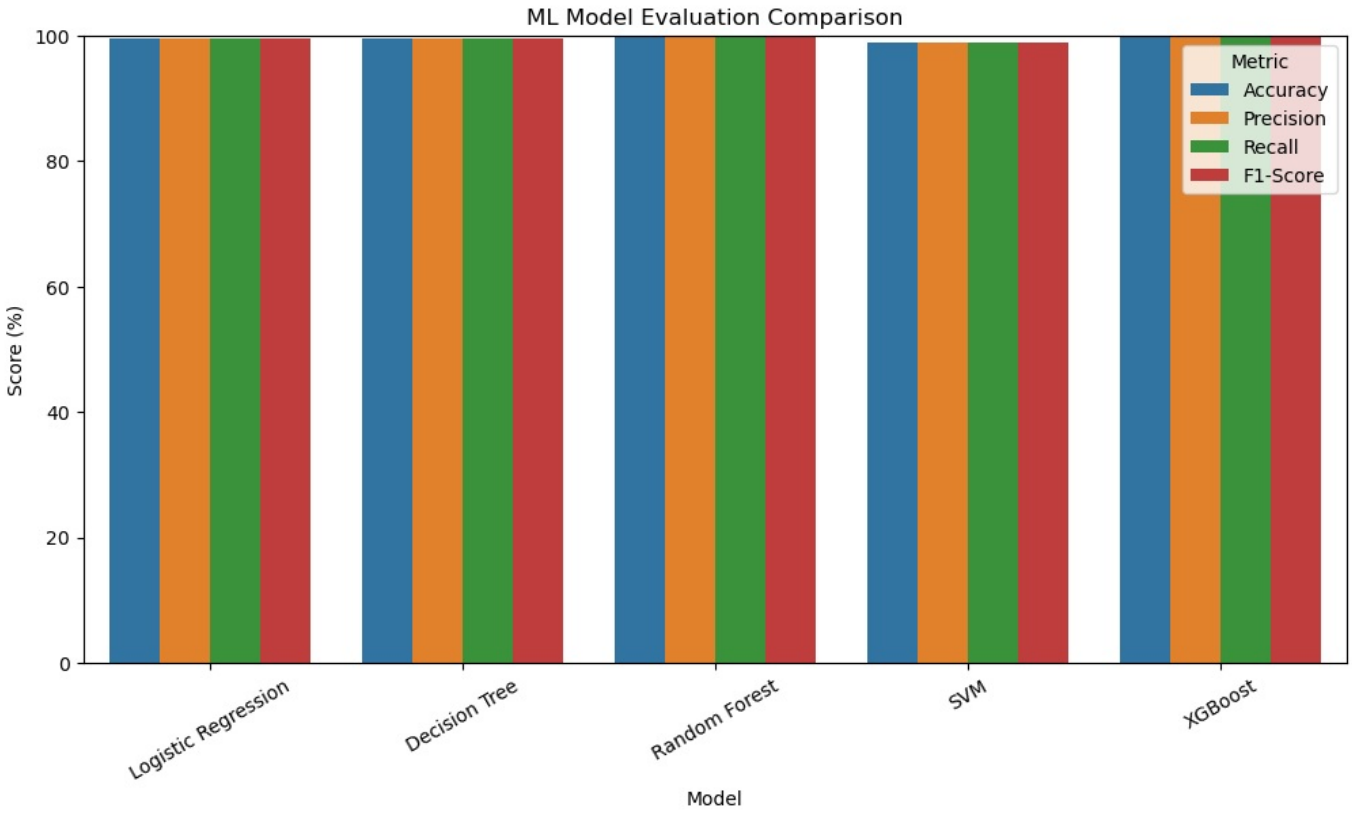
sns.barplot(
    data=metrics_long,
    x="Model",
    y="Score",
    hue="Metric"
)

plt.title("ML Model Evaluation Comparison")
plt.xlabel("Model")
plt.ylabel("Score (%)")

plt.ylim(0, 100)

plt.xticks(rotation=30)

plt.show()
```



In [40]:
```
model_predictions = {
    "Logistic Regression": y_pred_logistic,
    "Decision Tree": y_pred_dt,
    "Random Forest": y_pred_rf,
    "SVM": y_pred_svm,
    "XGBoost": y_pred_xgb
}
```

In [41]:
```
for model_name, predictions in model_predictions.items():

    cm = confusion_matrix(
        y_test,
        predictions
    )

    plt.figure(figsize=(7, 5))

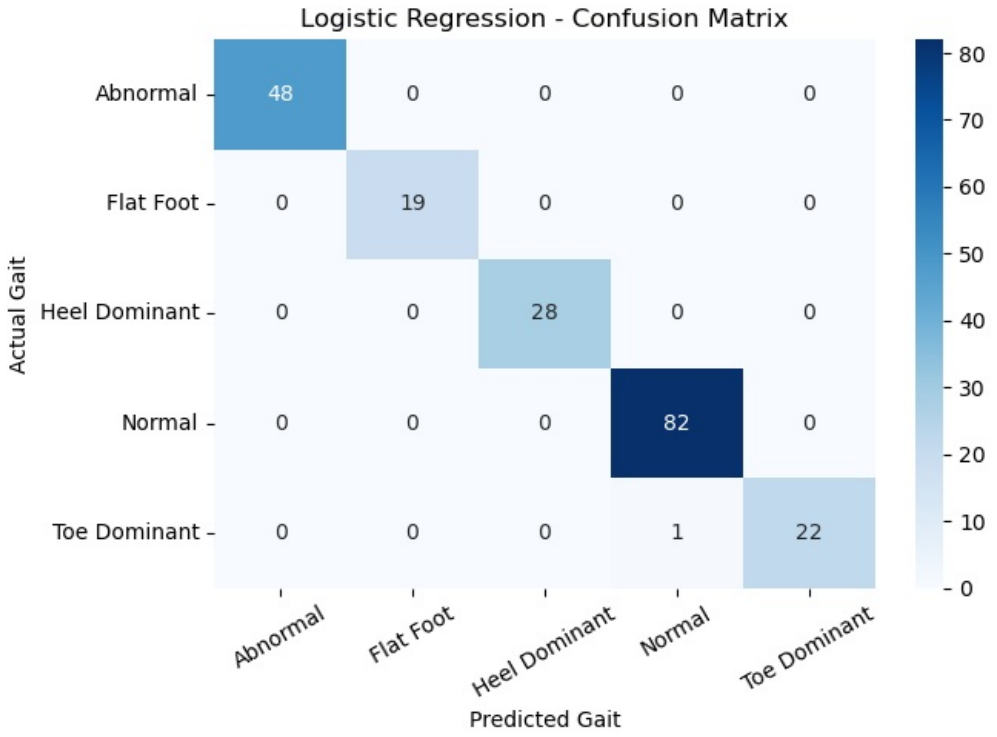
    sns.heatmap(
        cm,
        annot=True,
        fmt="d",
        cmap="Blues",
        xticklabels=label_encoder.classes_,
        yticklabels=label_encoder.classes_
    )

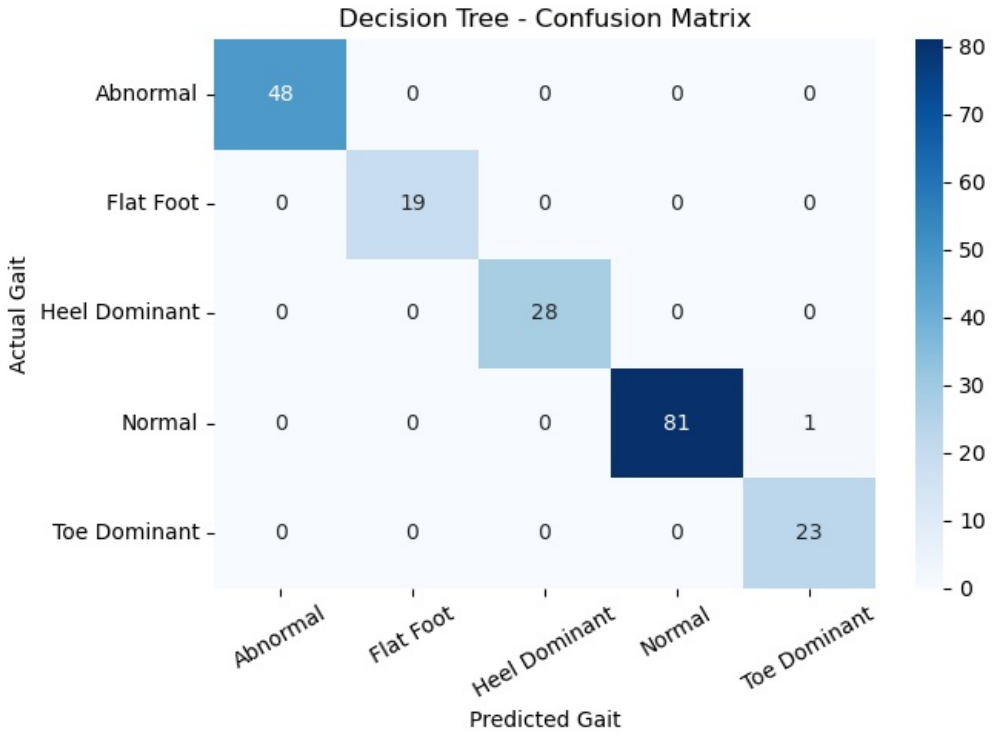
    plt.title(f"{model_name} - Confusion Matrix")

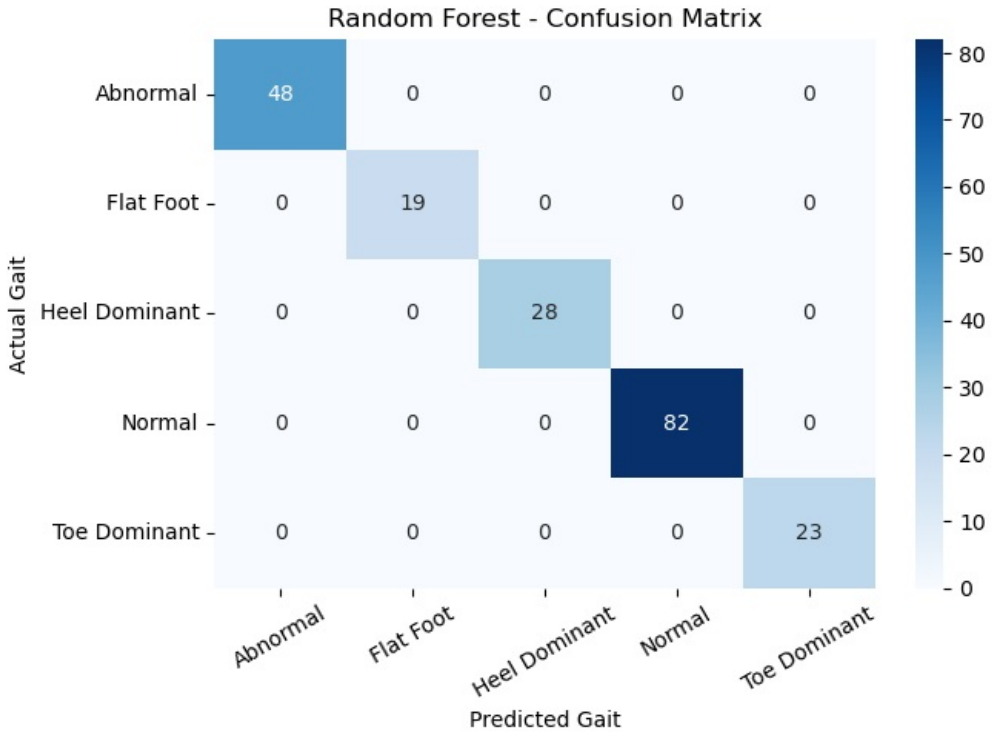
    plt.xlabel("Predicted Gait")
    plt.ylabel("Actual Gait")

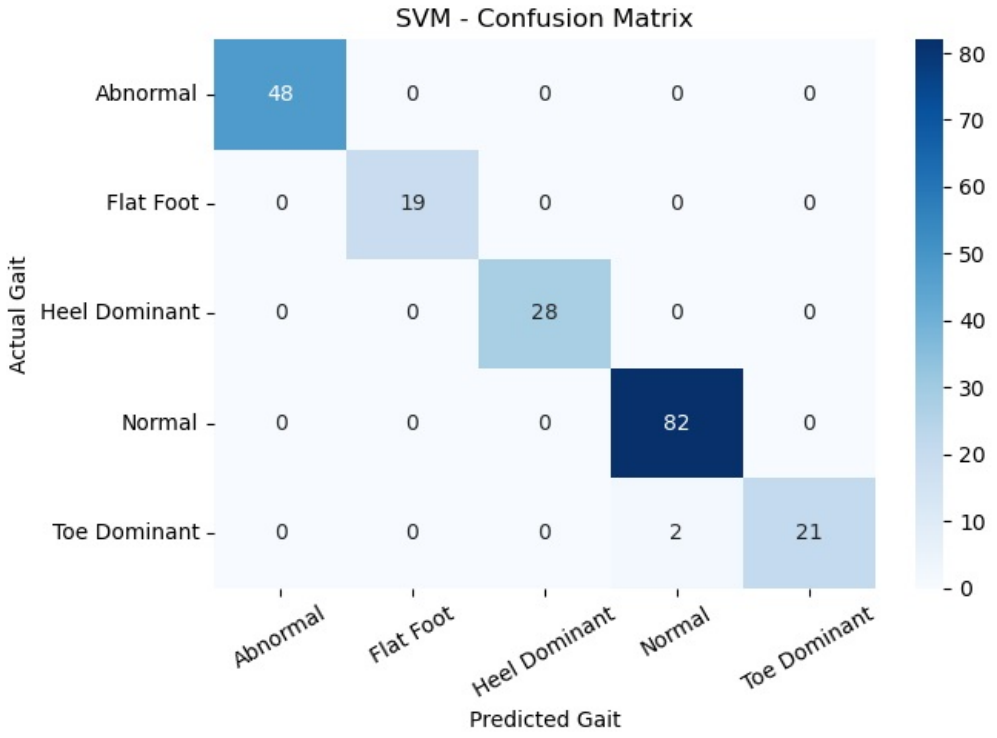
    plt.xticks(rotation=30)
    plt.yticks(rotation=0)

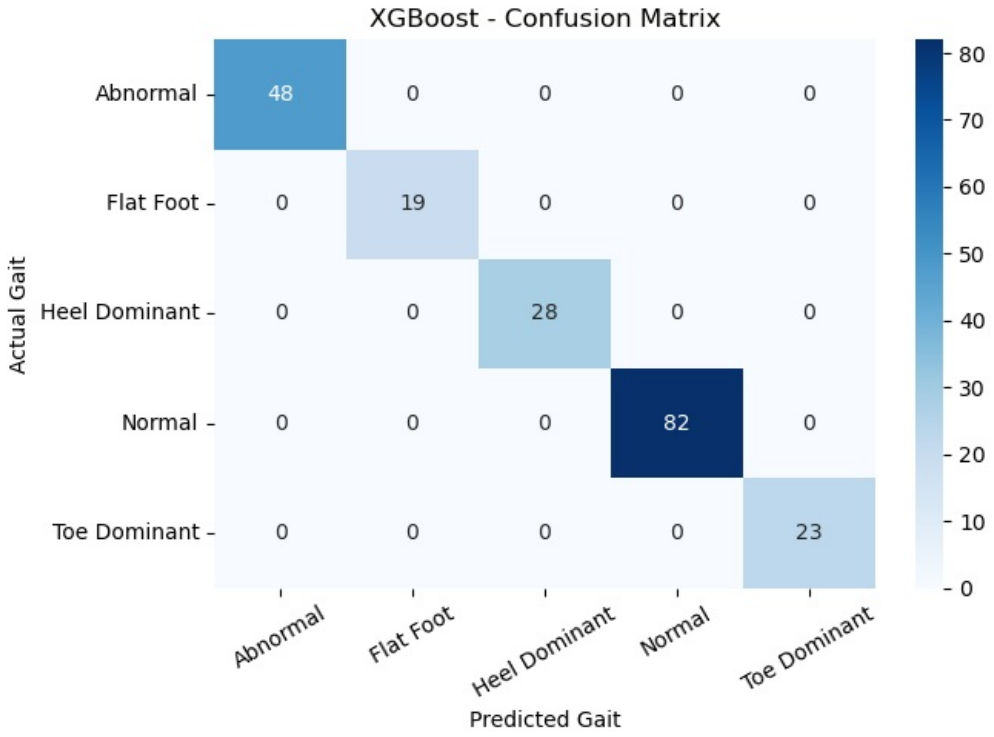
    plt.tight_layout()
    plt.show()
```


Logistic Regression - Confusion Matrix
Actual Gait
Abnormal
Flat Foot
Heel Dominant
Normal
Toe Dominant
48 0 0 0 0
0 19 0 0 0
0 0 28 0 0
0 0 0 82 0
0 0 0 1 22
Abnormal
Flat Foot
Heel Dominant
Normal
Toe Dominant
Predicted Gait
80
70
60
50
40
30
20
10
0


Decision Tree - Confusion Matrix
Actual Gait
Abnormal
Flat Foot
Heel Dominant
Normal
Toe Dominant
48 0 0 0 0
0 19 0 0 0
0 0 28 0 0
0 0 0 81 1
0 0 0 0 23
Abnormal
Flat Foot
Heel Dominant
Normal
Toe Dominant
Predicted Gait
80
70
60
50
40
30
20
10
0


Random Forest - Confusion Matrix
Abnormal
Flat Foot
Heel Dominant
Normal
Toe Dominant
Actual Gait
48 0 0 0 0
0 19 0 0 0
0 0 28 0 0
0 0 0 82 0
0 0 0 0 23
Abnormal
Flat Foot
Heel Dominant
Normal
Toe Dominant
Predicted Gait
80
70
60
50
40
30
20
10
0


SVM - Confusion Matrix
Abnormal
Flat Foot
Heel Dominant
Normal
Toe Dominant
Actual Gait
48 0 0 0 0
0 19 0 0 0
0 0 28 0 0
0 0 0 82 0
0 0 0 2 21
Abnormal
Flat Foot
Heel Dominant
Normal
Toe Dominant
Predicted Gait
80
70
60
50
40
30
20
10
0



In [42]:
```
best_predictions = model_predictions[best_model_name]
```

In [43]:
```
print(
    classification_report(
        y_test,
        best_predictions,
        target_names=label_encoder.classes_,
        zero_division=0
    )
)
```

```
              precision    recall  f1-score   support

    Abnormal       1.00      1.00      1.00        48
   Flat Foot       1.00      1.00      1.00        19
Heel Dominant       1.00      1.00      1.00        28
      Normal       1.00      1.00      1.00        82
 Toe Dominant       1.00      1.00      1.00        23

    accuracy                           1.00       200
   macro avg       1.00      1.00      1.00       200
weighted avg       1.00      1.00      1.00       200
```

In [44]:
```
trained_models = {
    "Logistic Regression": logistic_model,
    "Decision Tree": dt_model,
    "Random Forest": rf_model,
    "SVM": svm_model,
    "XGBoost": xgb_model
}

print("All trained models stored successfully!")
```

```
All trained models stored successfully!
```

In [ ]:
```
print("=" * 50)
print("       SMART SHOE GAIT ANALYSIS SYSTEM")
print("=" * 50)

# 1. USER INPUT

age = int(input("Enter Age (3-8): "))
gender = input("Enter Gender (Male/Female): ")

height = float(input("Enter Height (cm): "))
weight = float(input("Enter Weight (kg): "))

heel_pressure = float(
    input("Enter Heel Pressure (kPa): ")
)

midfoot_pressure = float(
    input("Enter Midfoot Pressure (kPa): ")
```

```
)

toe_pressure = float(
    input("Enter Toe Pressure (kPa): ")
)

walking_speed = float(
    input("Enter Walking Speed (m/s): ")
)

# 2. FEATURE CALCULATION

bmi = weight / ((height / 100) ** 2)

total_pressure = (
    heel_pressure +
    midfoot_pressure +
    toe_pressure
)

pressure_values = {
    "Heel": heel_pressure,
    "Midfoot": midfoot_pressure,
    "Toe": toe_pressure
}

dominant_zone = max(
    pressure_values,
    key=pressure_values.get
)

heel_ratio = heel_pressure / total_pressure
midfoot_ratio = midfoot_pressure / total_pressure
toe_ratio = toe_pressure / total_pressure


# 3. CREATE USER DATAFRAME

user_data = pd.DataFrame({

    "Age": [age],
    "Gender": [gender],
    "Height_cm": [height],
    "Weight_kg": [weight],
    "BMI": [bmi],

    "Heel_Pressure_kPa": [heel_pressure],
    "Midfoot_Pressure_kPa": [midfoot_pressure],
    "Toe_Pressure_kPa": [toe_pressure],

    "Total_Pressure_kPa": [total_pressure],

    "Dominant_Zone": [dominant_zone],

    "Walking_Speed_mps": [walking_speed],

    "Heel_Ratio": [heel_ratio],
    "Midfoot_Ratio": [midfoot_ratio],
    "Toe_Ratio": [toe_ratio]
})


# 4. ENCODING

user_encoded = pd.get_dummies(
    user_data,
    columns=[
        "Gender",
        "Dominant_Zone"
    ],
    drop_first=False
)

user_encoded = user_encoded.reindex(
    columns=X.columns,
    fill_value=0
)

# 5. SCALING

if best_model_name in [
    "Logistic Regression",
    "SVM"
```

```
]:

    user_input_for_model = scaler.transform(
        user_encoded
    )

else:

    user_input_for_model = user_encoded

# 6. PREDICTION

prediction_encoded = trained_models[
    best_model_name
].predict(
    user_input_for_model
)

predicted_gait = label_encoder.inverse_transform(
    prediction_encoded
)[0]

# 7. DISEASE / RISK / RECOMMENDATION

analysis_result = gait_output_mapping[
    predicted_gait
]

# 8. FINAL RESULT

print("\n")
print("=" * 50)
print("          SMART SHOE ANALYSIS RESULT")
print("=" * 50)

print(f"Patient Age       : {age}")
print(f"Gender            : {gender}")
print(f"Height            : {height} cm")
print(f"Weight            : {weight} kg")
print(f"BMI               : {bmi:.2f}")

print(f"\nHeel Pressure     : {heel_pressure} kPa")
print(f"Midfoot Pressure  : {midfoot_pressure} kPa")
print(f"Toe Pressure      : {toe_pressure} kPa")

print(f"\nTotal Pressure    : {total_pressure} kPa")
print(f"Dominant Zone     : {dominant_zone}")

print("\n---------------- RESULT ----------------")

print(f"Gait Type         : {predicted_gait}")

print(
    f"Disease           : "
    f"{analysis_result['Disease']}"
)

print(
    f"Risk Level        : "
    f"{analysis_result['Risk_Level']}"
)

print(
    f"Recommendation    : "
    f"{analysis_result['Recommendation']}"
)

print(f"\nModel Used        : {best_model_name}")

print("=" * 50)
```

```
================================================
       SMART SHOE GAIT ANALYSIS SYSTEM
================================================


================================================
          SMART SHOE ANALYSIS RESULT
================================================
Patient Age       : 5
Gender            : male
Height            : 112.0 cm
Weight            : 20.0 kg
BMI               : 15.94

Heel Pressure     : 180.0 kPa
Midfoot Pressure  : 120.0 kPa
Toe Pressure      : 30.0 kPa

Total Pressure    : 330.0 kPa
Dominant Zone     : Heel

---------------- RESULT ----------------
Gait Type         : Heel Dominant
Disease           : High Heel Pressure
Risk Level        : Medium
Recommendation    : Use cushioned footwear.

Model Used        : Random Forest
================================================
```

In [61]:
```
results_percentage
```

Out[61]:

	Model	Accuracy	Precision	Recall	F1-Score
0	Logistic Regression	99.5	99.51	99.5	99.50
1	Decision Tree	99.5	99.52	99.5	99.50
2	Random Forest	100.0	100.00	100.0	100.00
3	SVM	99.0	99.02	99.0	98.98
4	XGBoost	100.0	100.00	100.0	100.00

In [ ]:
```

```